MEMORIA ACTIVIDAD 1

1. Entorno y procesos seleccionados

La actividad se ha realizado en Windows 11con Java JDK 23 e IntelliJ IDEA 2026. Los comandos se lanzan mediante cmd /c porque ver y type son comandos internos del intérprete de Windows. Se han seleccionado cuatro procesos: dos correctos y dos pruebas controladas de error.

Nº	Proceso	Comando	Tipo
1	proceso1_bat	cmd /c <ruta>\resources\fichero.bat (directorio de trabajo: process)	Correcto
2	proceso2_ver	cmd /c ver	Correcto
3	proceso3_error_recurso	cmd /c type archivo_que_no_existe.txt	Error controlado: recurso inexistente
4	proceso4_error_comando	comando_que_no_existe --opcion	Error controlado: comando inexistente

El fichero fichero.bat (carpeta resources) contiene: MKDIR NUEVO, CD NUEVO, ECHO FICHERO CREADO > hola.txt, DIR, CDDD y ECHO FIN COMANDOS. El comando CDDD no existe a propósito, para generar un mensaje en la salida de error.

2. Decisiones adoptadas

- Array de ProcessBuilder y bucle for. Los cuatro procesos se configuran en un array y se lanzan uno a uno, de forma secuencial y sin hilos, evitando repetir código. En cada vuelta, start() inicia el proceso y waitFor() espera a que termine y devuelve su código de salida.
- Argumentos y directorio de trabajo. Los procesos 2 y 3 reciben argumentos. El proceso 1 usa directory() con la carpeta process, de modo que NUEVO y hola.txt se crean allí y no en la raíz del proyecto. Como cambia el directorio de trabajo, se pasa la ruta absoluta del .bat.
- Registros separados mediante redirección. redirectOutput() y redirectError() envían cada flujo a un fichero distinto dentro de la carpeta logs (<proceso>_salida.log y <proceso>_error.log). Así nunca se mezclan, y al no leer los flujos a mano no hay riesgo de bloqueo por buffer lleno ni hace falta programación multihilo.
- Gestión de excepciones. El try/catch está dentro del bucle: IOException cubre que el proceso no pueda iniciarse (proceso 4) e InterruptedException una espera interrumpida. En ambos casos se muestra un mensaje y la aplicación continúa con el siguiente proceso.
- Recursos. La redirección la gestiona ProcessBuilder, por lo que el programa no mantiene flujos de lectura abiertos que deba cerrar.
- Resumen por consola. Tras cada ejecución se imprime una línea con el proceso y su comando, si se inició (SI/NO), el código de salida (o «no disponible») y los dos ficheros de registro.

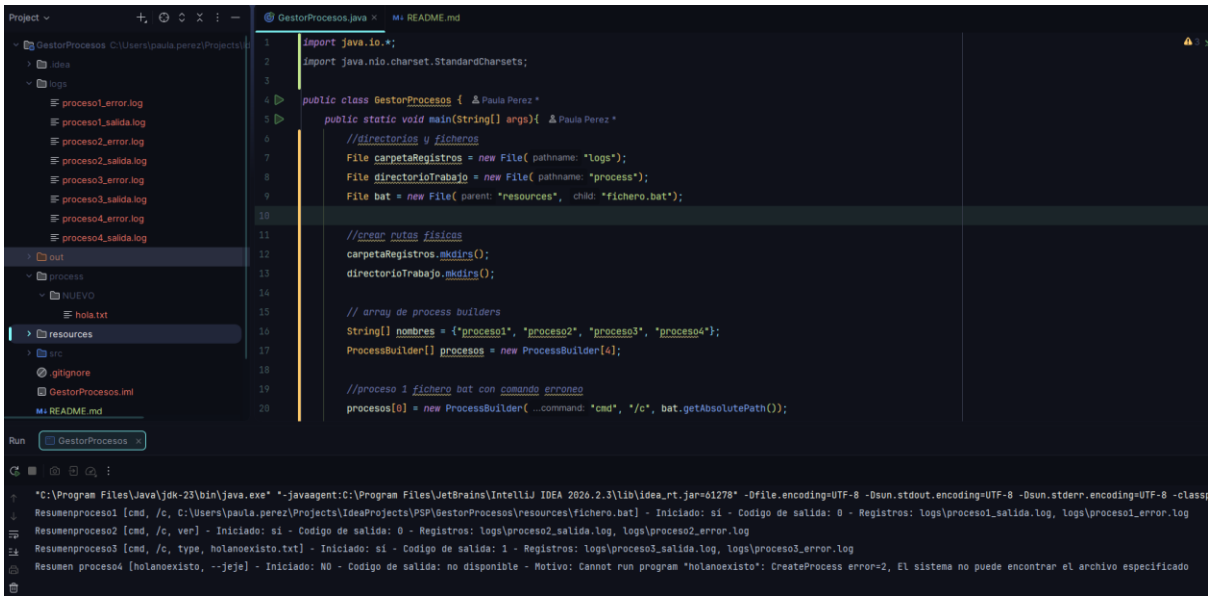
3. Pruebas y resultados

- Proceso 1: código 0. En proceso1_salida.log aparecen los comandos y el listado DIR; en proceso1_error.log, el mensaje de CDDD. Se crea process\NUEVO\hola.txt. El código es 0 aunque CDDD falle, porque un .bat devuelve el código de su último comando.

- Proceso 2: código 0. La versión de Windows queda en proceso2_ salida.log y el registro de error queda vacío.
- Proceso 3: se inicia y termina con código 1. El mensaje “no se encuentra el archivo” queda en proceso3_error.log y el de salida queda vacío.
- Proceso 4: no se inicia y se queda sin código de salida. La IOException se captura y la aplicación termina con normalidad.

Si el programa se ejecuta dos veces, MKDIR NUEVO falla porque la carpeta ya existe y el mensaje queda en el registro de error; para repetir la prueba se borra process\NUEVO.

4. Evidencias de funcionamiento



5. Conclusiones

La aplicación cumple los requisitos: lanza los procesos de forma secuencial, registra su código de salida, separa salida y error en ficheros distintos y continúa tras los errores controlados. Lo más relevante fue distinguir un proceso que no se inicia (excepción) de uno que se inicia y termina con error (código distinto de 0), y comprobar que el código de salida de un .bat depende de su último comando.